Kubernetes 101: The What and the Why

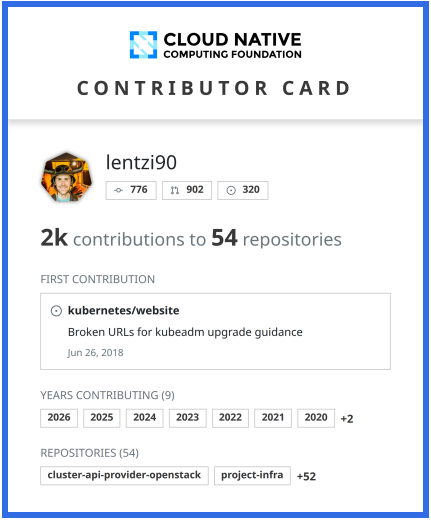
Lennart Jern

Ericsson Software Technology

October 21, 2026

About me

- Lennart Jern, Software Engineer at Ericsson
- Open source maintainer: Cluster API Provider OpenStack, Metal3
- Cluster API release team
- GitHub: lentzi90



Kubernetes 101: The What and the Why

- Goal: understand *why* Kubernetes looks the way it does
- If you know the problem, the design is much easier to remember than a list of resource types
- No prior Kubernetes knowledge needed

The Problem: Running Software

Before containers:

- One application per server: wasteful, slow to provision
- "Works on my machine": dependencies and OS differences
- Manual deployments, snowflake servers

Virtual machines helped with isolation, but they are heavy: a full OS per application.

Containers: Packaging and Isolation

- A container image bundles the app **and** its dependencies
- Runs the same on a laptop, a server, or in the cloud
- Isolated using Linux kernel features (namespaces, cgroups), not a full OS
- Lightweight: starts in seconds, many per machine

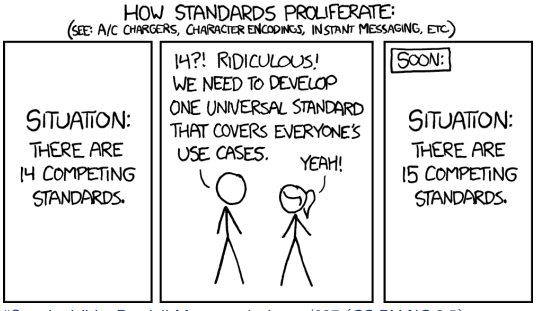
Important: a container is just a (isolated) process.

Containers Are Not Enough

You now have hundreds of containers across dozens of machines. Who...

- decides **where** each one runs?
- **restarts** it when it crashes, or when the machine dies?
- **scales** it up when load increases?
- lets other services **find** it when its address keeps changing?
- **upgrades** it without downtime?

This is the job of a *container orchestrator*.



"Standards" by Randall Munroe, xkcd.com/927 (CC BY-NC 2.5)

What is Kubernetes?

- Open source container orchestrator, originally from Google (inspired by their internal system, Borg)
- Donated to the CNCF in 2015, now the de-facto standard
- Greek for "helmsman" (hence the logo, and the K8s abbreviation: K + 8 letters + s)
- One API to run workloads on laptops, data centers, public clouds, or edge sites

Not a PaaS: it gives you building blocks, not a finished platform.



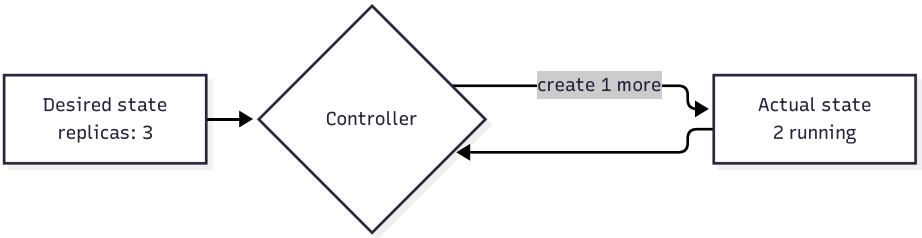
The Core Idea: Desired State

Imperative: "Start 3 copies of this container on server A, B and C."

Declarative: "I want 3 copies of this container running."

- You describe *what* you want, not *how* to get there
- Kubernetes continuously works to make reality match your description
- Everything you do is storing desired state in the API

Control Loops (Reconciliation)

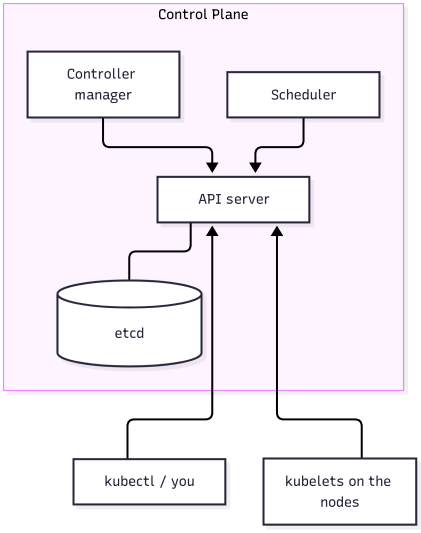


- A controller *observes* actual state, *compares* it with desired state, and *acts* on the difference
- Repeat forever
- Crash, node failure, manual deletion: the loop just notices the difference and fixes it
- This is why Kubernetes is called **self-healing**

Control Plane: The Brain

- **API server**: the only entry point. Everyone (you, controllers, kubelets) talks through it
- **etcd**: the database. Stores all desired state
- **Scheduler**: picks a node for each new Pod
- **Controller manager**: runs the control loops (Deployments, Nodes, . . .)

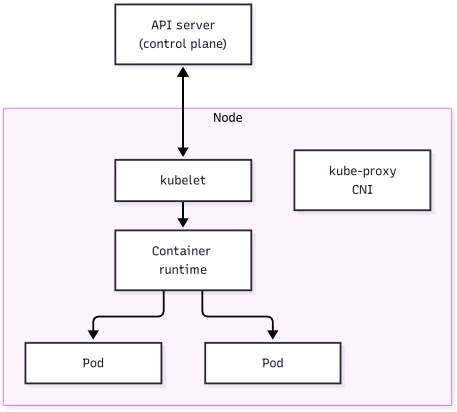
Why this split? Each component does one small job, and communicates only via the API. Easy to replace, extend, and reason about.



Nodes: The Muscle

- A node is a machine (physical or virtual) that runs workloads
- **kubelet**: agent on the node, makes sure the containers it was told to run are running
- **Container runtime** (e.g. containerd): actually runs the containers
- **Kube-proxy / CNI**: networking

Nodes are *cattle, not pets*: if one dies, the work moves to another.



Pods: The Smallest Unit

- A Pod is one or more containers that are **scheduled together**
- Containers in a Pod share network (localhost) and can share storage
- Typical case: one main container; sidecars for logging, proxies, etc.
- Pods are **ephemeral**: they get replaced, not repaired

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: hello
spec:
  containers:
    - name: web
      image: nginx:1.27
```

Deployments

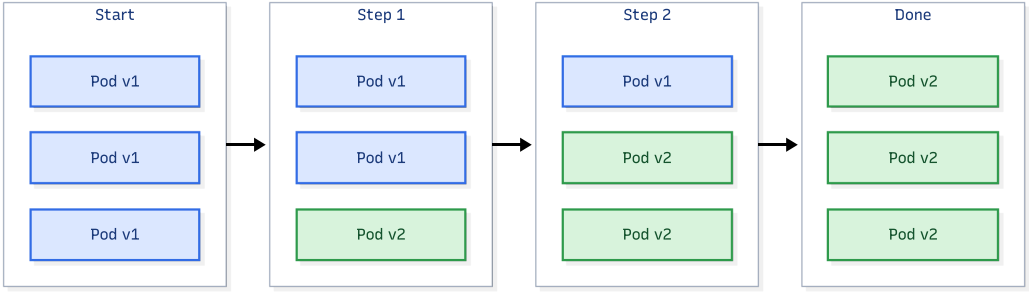
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello
spec:
  replicas: 3
  selector:
    matchLabels:
      app: hello
  template:
    metadata:
      labels:
        app: hello
    spec:
      containers:
        - name: web
          image: nginx:1.27
```

"Keep 3 Pods like this running, forever."

- the desired state: `replicas` + `image`
- `template`: what each Pod looks like
- `selector` + labels: which Pods belong to this Deployment

Deployments: Rolling Updates

- Change the image version in the Deployment: that's it
- Kubernetes gradually creates new Pods and removes old ones
- Traffic keeps flowing, old version stays until the new one is ready
- Something wrong? Roll back to the previous version

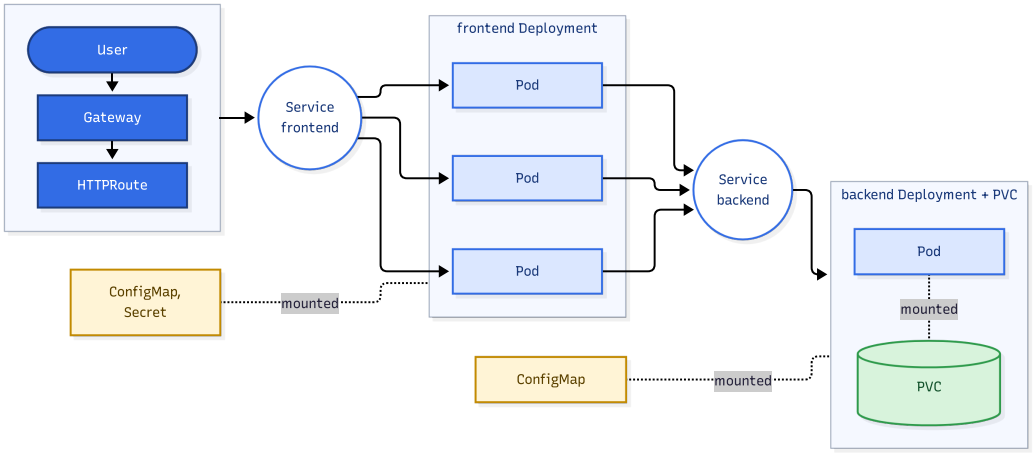


Pods are immutable: we don't patch them, we replace them.

Demo time!

Declaratively, from one Deployment to a full application

Putting It All Together



Takeaways

- Containers solve packaging, Kubernetes solves running them **at scale**
- **Declarative desired state + control loops** is the core idea
- Pods are ephemeral; Deployments keep them alive and updated; Services make them reachable
- Configuration, storage, and ingress are separated from the application on purpose

Questions?

Thank you!

Links

- Smooth sailing with Kubernetes
- 12-factor app: https://12factor.net
- My GitHub profile: lentzi90
- Metal3: https://metal3.io
- Cluster API: https://cluster-api.sigs.k8s.io